PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Graduação em Engenharia de Computação - Disciplina de Sistemas Reconfiguráveis

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE UM CIRCUITO LÓGICO COMBINACIONAL UTILIZANDO VHDL

Belo Horizonte
19 de outubro de 2023

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE UM CIRCUITO LÓGICO COMBINACIONAL UTILIZANDO VHDL

Trabalho desenvolvido como parte dos projetos
da disciplina de Sistemas Reconfiguráveis

Professor: Antônio Hamilton Magalhaes

Belo Horizonte 2023

Lista de Figuras

Figura 1: Diagrama do MUX.

Figura 2: Definição das bibliotecas MUX.

Figura 3: Definição da Entidade contendo entradas e saídas do MUX em VHDL.

Figura 4: Definição da arquitetura do MUX.

Figura 5: Definição das bibliotecas.

Figura 6: Definição da Entidade contendo entradas e saídas em VHDL.

Figura 7: Definição da arquitetura.

Figura 8: Configurações do Carry.

Figura 9: Seção da arquitetura responsável pelas operações.

Figura 10: Configurações das saídas e o final da arquitetura.

Figura 11: Simulação da operação AND.

Figura 12: Simulação da operação OR.

Figura 13: Simulação da operação XOR.

Figura 14: Simulação da operação COM.

Figura 15: Simulação da operação INC.

Figura 16: Simulação da operação DEC.

Figura 17: Simulação da operação ADD.

Figura 18: Simulação da operação add com carry no primeiro nibble.

Figura 19: Simulação da operação SUB.

Figura 20: Simulação da operação SWAP.

Figura 21: Simulação da operação CLR.

Figura 22: Simulação da operação RR.

Figura 23: Simulação da operação RL.

Figura 24: Simulação da operação BS.

Figura 25: Simulação da operação BC.

Figura 26: Simulação da operação PASS A.

Figura 27: Simulação da operação PASS B.

Lista de Tabelas

Tabela 1 - Bits de Seleção por operação realizada na ULA.

Tabela 2 – Entradas por resultado esperado em cada operação.

SUMÁRIO

I INTRODUÇÃO 6

II METODOLOGIA ADDR_MUX 6

II.I SEÇÃO 1 6

II.II SEÇÃO 2 7

II.III SEÇÃO 3 7

III METODOLOGIA ALU 7

III.I SEÇÃO 1 7

III.II SEÇÃO 2 8

III.III SEÇÃO 3 11

IV CONCLUSÃO 23

V REFERÊNCIAS 23

I - INTRODUÇÃO

O VHDL é uma linguagem de descrição de hardware, amplamente utilizada para projetar circuitos digitais e sistemas integrados. Ela oferece uma maneira eficaz de modelar, simular e sintetizar dispositivos eletrônicos, permitindo o desenvolvimento de hardware de forma eficiente e com alto grau de flexibilidade.

Neste trabalho, serão apresentados dois projetos independentes: addr_mux e uma alu (unidade lógica aritmética). Ambos os projetos serão desenvolvidos em VHDL, com ênfase na descrição do código, simulação e análise dos resultados, de acordo com as especificações fornecidas.

Cada projeto será implementado de forma totalmente combinacional, o que significa que não envolverá elementos de armazenamento, como latches ou flip-flops. Todas as entradas e saídas dos projetos serão do tipo std_logic ou std_logic_vector, seguindo as convenções de design comuns em linguagens de descrição de hardware.

O primeiro projeto, addr_mux, e o segundo projeto, alu, serão abordados separadamente em capítulos distintos no relatório. Com base nesses princípios, este trabalho visa explorar a capacidade do VHDL na modelagem de circuitos digitais, fornecendo uma visão abrangente do processo de desenvolvimento de projetos de hardware combinacionais, com ênfase na ULA, uma unidade fundamental em processadores. A ULA será projetada com 16 bits de operações em palavras de 8 bits, incluindo bits de carry-in e carry-out. A metodologia empregada segue as especificações fornecidas pelos professores, garantindo um projeto eficiente e preciso.

II - METODOLOGIA ADDR_MUX

A metodologia para projetar o multiplexador com saída de 9 bits, conforme as especificações fornecidas, envolve a criação de um circuito lógico que selecione a saída de endereço com base nas entradas fornecidas.

III.I - SEÇÃO 1 – Entradas e Saídas

Primeiro, é importante entender as entradas e saídas do multiplexador, bem como suas condições de funcionamento.

- rp_in[1..0]: Entrada de seleção de banco para endereçamento direto.
- dir_addr_in[6..0]: Entrada para endereçamento direto.
- irp_in: Entrada de seleção de banco para endereçamento indireto.
- ind_addr_in[7..0]: Entrada para endereçamento indireto.
- abus_out[8..0]: Saída de endereço.

III.II - SEÇÃO 2 – Diagrama Equivalente

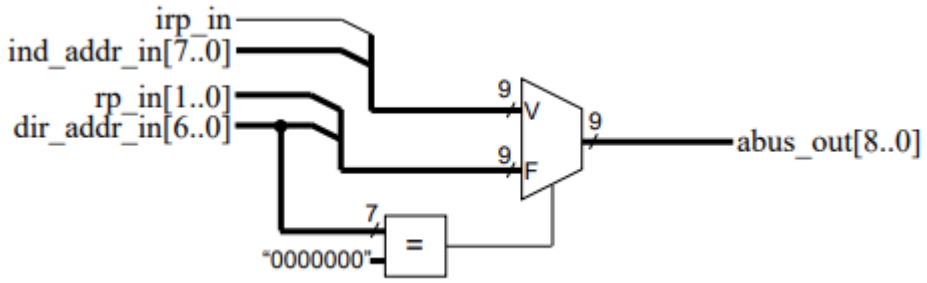

Figura 1: Diagrama do MUX.

III.II - SEÇÃO 3 – Descrição do mux em VHDL

```
library IEEE;
use IEEE.STD LOGIC 1164.ALL;
```

Figura 2: Definição das bibliotecas MUX.

```
entity Multiplexador is
    Port ( rp_in : in STD_LOGIC_VECTOR(1 downto 0);
           dir_addr_in : in STD_LOGIC_VECTOR(6 downto 0);
           irp_in : in STD_LOGIC;
           ind_addr_in : in STD_LOGIC_VECTOR(7 downto 0);
           abus_out : out STD_LOGIC_VECTOR(8 downto 0));
end Multiplexador;
```

Figura 3: Definição da Entidade contendo entradas e saídas do MUX em VHDL.

```
architecture Behavioral of Multiplexador is
begin
    process (rp_in, dir_addr_in, irp_in, ind_addr_in)
    begin
        if dir_addr_in = "0000000" then
            abus_out <= irp_in & ind_addr_in; -- Concatenação de irp_in e ind_addr_in
        else
            abus_out <= rp_in & dir_addr_in; -- Concatenação de rp_in e dir_addr_in
        end if;
    end process;
end Behavioral;
```

Figura 4: Definição da arquitetura do MUX.

III - METODOLOGIA ALU

A metodologia utilizada no projeto da ULA foi elaborada com base na especificação do projeto fornecida pelos professores da disciplina de Sistemas Reconfiguráveis. A especificação previa 16 bits de operações e duas entradas A e B, ambas de 8 bits, além da entrada e saída de *carry.*

III.I - SEÇÃO 1: Operações da ULA e Resultados Esperados:

Na especificação do projeto foram fornecidas as operações equivalentes à cada combinação dos bits de seleção, como será visto na Tabela 1. Na Tabela 2 se encontram alguns resultados esperados para cada operação com as entradas que

utilizamos nas simulações, porém, mais detalhes sobre as simulações estarão relatados nas seções 2 e 3.

OPCODE	OP
0000	AND
0001	OR
0010	XOR
0011	COM
0100	INC
0101	DEC
0110	ADD
0111	SUB
1000	SWAP
1001	CLR
1010	Rotação para direita
1011	Rotação para esquerda
1100	BS
1101	BC
1110	By-pass A
1111	By-pass B

Tabela 1: Bits de Seleção por operação realizada na ALU.

III.II - SEÇÃO 2: Descrição da ALU em VHDL

Inicialmente, colocamos as bibliotecas que serão utilizadas no código.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

Figura 5: Definição das bibliotecas.

Após a definição das bibliotecas é feita a descrição da ENTIDADE e das entradas e saídas, como será demonstrado na FIGURA 6.

```vhdl
ENTITY alu IS
PORT (
        a_in, b_in : IN STD_LOGIC_VECTOR (7 DOWNTO 0); -- entrada "a" e "b" dos dados
        c_in: IN STD_LOGIC; -- entrada de carry (usada nas operações de RR e RL)
        op_sel: IN STD_LOGIC_VECTOR (3 DOWNTO 0); -- entrada de seleção de operação
        bit_sel: IN STD_LOGIC_VECTOR (2 DOWNTO 0); -- entrada de seleção de bit (usada nas operações de BS e BC)
        r_out: OUT STD_LOGIC_VECTOR (7 DOWNTO 0);
        c_out: OUT STD_LOGIC; --saida do resultado
        dc_out: OUT STD_LOGIC; --saida do carry/barrow
        z_out: OUT STD_LOGIC -- bit_sel quando é utilizado BS ou BC, nas outras entradas é 0
);
END ENTITY;
```

Figura 6: Definição da Entidade contendo entradas e saídas em VHDL.

Na Figura 6, o tipo STD_LOGIC_VECTOR é utilizado para conter as palavras de 8 bits nas entradas A e B, e o opcode de 4 bits. Dois STD_LOGIC de 1 bit são utilizados para o *carry in*, *carry out* e z_out (indicador de zero).

Em VHDL deve-se descrever uma arquitetura que será utilizada na simulação e possivelmente sintetizada em algum dispositivo FPGA para teste. O código em VHDL para cada operação pode ser observado na Figura 3.

```vhdl
ARCHITECTURE arch OF alu IS

    SIGNAL a_temp, aux: STD_LOGIC_VECTOR (8 downto 0); -- temporario de a/saida de 9 bits da ula
    SIGNAL temp_sum : INTEGER; --somador temporario
    SIGNAL bit_sel_temp : STD_LOGIC_VECTOR(7 DOWNTO 0); -- bit select temporario que sera utilizado no BS E BC
    SIGNAL z_temp : STD_LOGIC; -- temporororario que corresponde ao bit_sel quando é utilizado BS ou BC, nas outras entradas é 0
```

Figura 7: Definição da arquitetura.

```vhdl
BEGIN

        a_temp <= '0' & a_in;

    WITH bit_sel SELECT
        bit_sel_temp <=  "00000001" WHEN "000",
                         "00000010" WHEN "001",
                         "00000100" WHEN "010",
                         "00001000" WHEN "011",
                         "00010000" WHEN "100",
                         "00100000" WHEN "101",
                         "01000000" WHEN "110",
                         "10000000" WHEN "111";
```

Figura 8: Configurações para BS e BC.

Na figura 8, utilizamos um bit_sel_temp para pegar o valor temporário em bit que foi armazenado nele e retorna à posição do vetor em 8.

```
WITH op_sel SELECT
    aux <='0' & (a_in AND b_in) WHEN "0000", -- AND
          '0' & (a_in OR b_in) WHEN "0001", -- OR
          '0' & (a_in XOR b_in) WHEN "0010", -- XOR
          '0' & (NOT a_in) WHEN "0011", -- COM
          ('0' & a_in) + 1 WHEN "0100", -- INC
          ('0' & a_in) - 1 WHEN "0101", -- DEC
          ('0' & a_in) + ('0' & b_in) WHEN "0110", -- ADD
          ('0' & a_in) - ('0' & b_in) WHEN "0111", -- SUB
          '0' & a_in(3 downto 0) & a_in(7 downto 4) WHEN "1000", -- SWAP
          '0' & "00000000" WHEN "1001", -- CLR
          a_temp(0) & c_in & a_temp(7 DOWNTO 1) WHEN "1010", --RR
          a_temp(7) & a_temp(6 DOWNTO 0) & c_in WHEN "1011",--RL
          '0' & bit_sel_temp OR a_in WHEN "1100", -- BS
          '0' & (NOT bit_sel_temp) AND a_in WHEN "1101", -- BC
          '0' & a_in WHEN "1110", -- PASS A
          '0' & b_in WHEN "1111"; -- PASS B
```

Figura 9: Seção da arquitetura responsável pelas operações.

Nas primeiras linhas estão as operações AND, OR, XOR e NOT listadas. Um bit em ‘0’ é concatenado mais à esquerda para completar 9 bits do auxiliar.

Esse bit adicional concatenado ao auxiliar é utilizado para obter o *carry out* nas operações, como por exemplo nas operações de ADD, onde o *overflow* da operação acaba sendo registrado no bit mais significativo, e esse bit é separado para a saída *carryout*.

No caso das operações, Rotação para Direita (RR), Rotação para Esquerda (RL) o vetor é separado em porções e se insere o bit mais significativo (ou menos) à direita (ou esquerda) a depender da operação.

Além disso, um bit em ‘0’ é também concatenado para completar o tamanho do ‘aux’ de 9 bits. Esse bit só não é utilizado quando o *carry in* é parte da rotação.

No caso da operação CLR, é utilizado para limpar o processo, já o BS e o BC ajustam e limpa respectivamente o bit apontado por bit_sel.

```
    r_out <= aux(7 downto 0);

    WITH op_sel SELECT
          dc_out <= aux(4) WHEN "0110",
                    NOT aux(4) WHEN "0111",
                    '0' WHEN OTHERS;

    WITH op_sel SELECT
           c_out <= aux (0) WHEN "1010",
                    aux (7) WHEN "1011",
                    aux (8) WHEN "0110" | "0111",
                    aux (8) WHEN OTHERS;


    WITH bit_sel SELECT
        z_temp <=   a_in(0) WHEN "000",
                    a_in(1) WHEN "001",
                    a_in(2) WHEN "010",
                    a_in(3) WHEN "011",
                    a_in(4) WHEN "100",
                    a_in(5) WHEN "101",
                    a_in(6) WHEN "110",
                    a_in(7) WHEN "111";

          z_out <=  z_temp WHEN op_sel = "1100" OR op_sel = "1101" ELSE
                   '1' WHEN aux(7 DOWNTO 0) = "00000000" ELSE
                   '0';



END arch;
```

Figura 10: Configurações das saídas e o final da arquitetura.

Na figura 10 podemos perceber as configurações das saídas, o uso do SIGNAL auxiliar como um receptor do resultado + carry. Isso foi necessário devido ao fato de que, na especificação do projeto, algumas operações tinham como carry out o bit mais à direita ou mais à esquerda da entrada, e assim, para obtê-los, foi necessário separar o sinal como observado

III.III - SEÇÃO 3: Simulações e testes no Quartus II:

Por fim, foram realizadas simulações através do software Quartus II. Em cada simulação, formas de onda representavam níveis lógicos em cada entrada e a simulação fornecia os níveis lógicos das saídas. Os valores de entrada utilizados e a saída esperada em cada um deles está na Tabela 2.

Simulação e Operação	**Entrada A**	**Entrada B**	**Saída Esperada**
AND	0000 0001 0000 0000	1111 1111 0000 0000	R= 00000001 Z=0 R= 00000000 Z=1
OR	0000 0001 0000 0000	1111 1111 0000 0000	R= 11111111 Z=0 R= 00000000 Z=1

XOR	0000 0001 0000 0000	1111 1111 0000 0000	R = 11111110 Z=0 R = 00000000 Z=1
COM	0000 0000 1111 1111	--	R = 11111111 Z=0 R = 00000000 Z=1
INC	0000 0001	--	R = 00000010 Z=0
DEC	0000 0001	--	R = 00000000 Z=1
ADD	0000 0001	1111 1111	R = 0000 0001 Z=1 e C=1
SUB	0000 1111	0000 1001	R = 00000110 Z=0, C=1 e DC=1
SWAP	0000 1111	1111 0000	R = 1111 0000
CLR	1011 1010	--	R = 00000000 Z = 1
Rotação pra direita	1111 1110	--	R = 11111111 C = 1
Rotação pra esquerda	0111 1111	--	R = 11111111 C = 1
BS	1010 1010	--	R = 10101010 Z = 1
BC	1010 1010	--	R = 10101000 Z = 1
pass A	1010 1010	--	R = 1010 1010 Z = 0
pass B	--	1111 1100	R = 11111100 Z = 0

Tabela 2: Indica a simulação, a operação, os bits na entrada A, B e os bits esperados na saída.

Na Tabela 2 estão listados os resultados esperados para as duas simulações realizadas no trabalho. O resultado será mais detalhado nas figuras correspondentes às simulações na Seção 3.

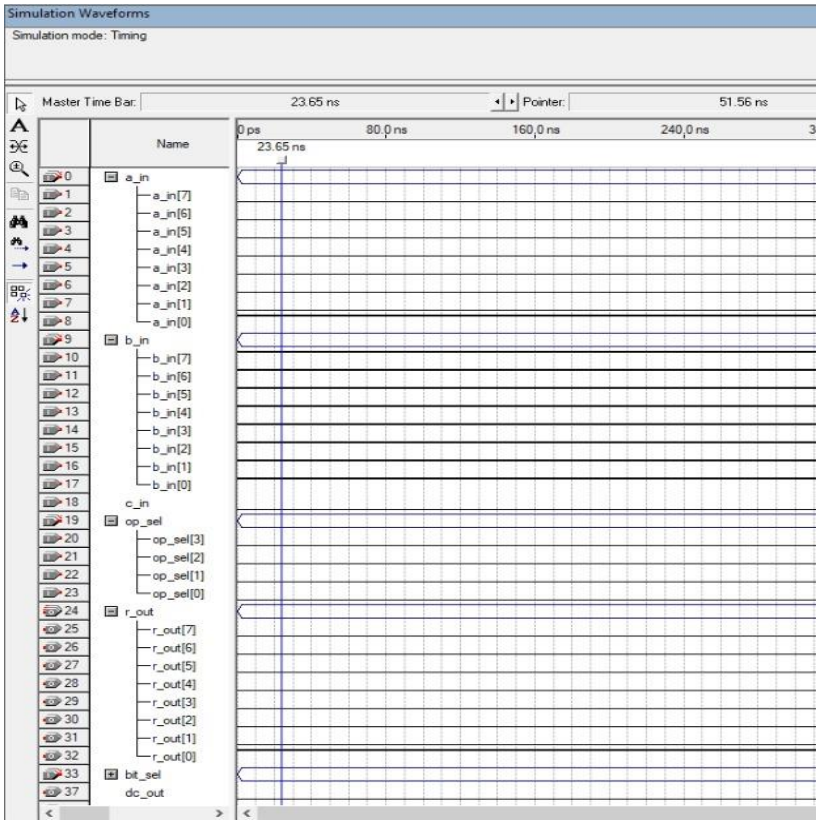

Figura 11: Simulação da operação AND.

A Figura 11 retrata a operação AND. No software Quartus II com entrada A = 0000 0001 e B = 1111 1111, o resultado foi R = 0000 0001 com z_out = 0. Quando a saída r for 0 o z_out = 1.

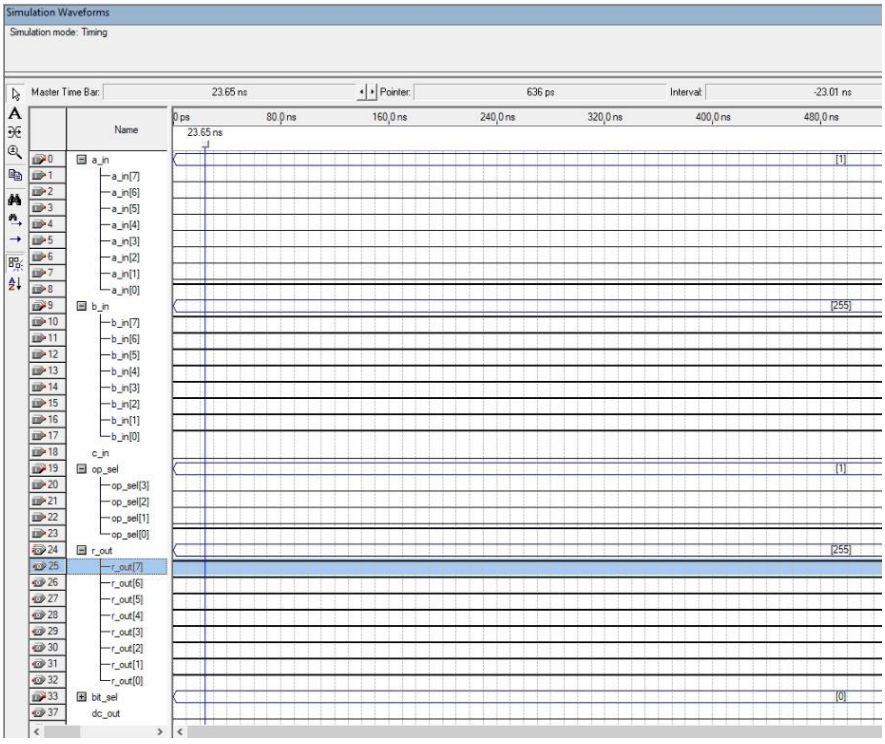

Figura 12: Simulação da operação OR

A Figura 12 retrata a operação OR. No software Quartus II com entrada A = 0000 0001 e B = 1111 1111, o resultado foi R = 1111 1111 com z_out = 0. Quando a saída r for 0 o z_out = 1.

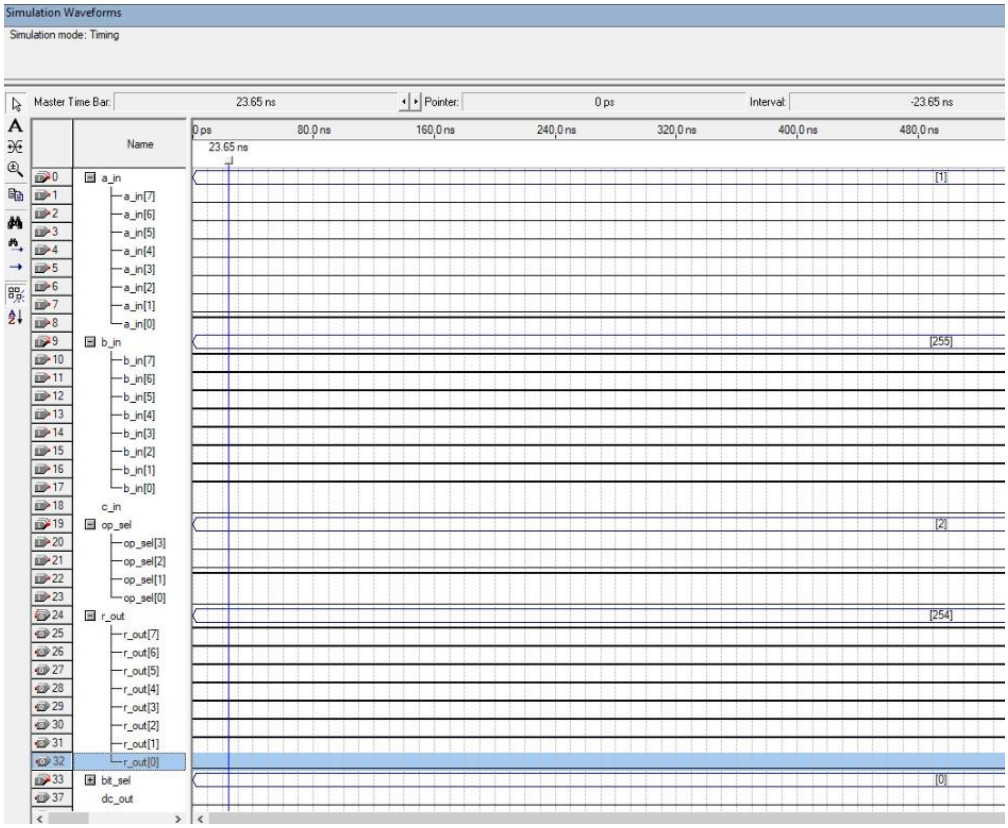

Figura 13: Simulação da operação XOR.

A Figura 13 retrata a operação XOR. No software Quartus II com entrada A = 0000 0001 e B = 1111 1111, o resultado foi R = 1111 1110 com z_out = 0. Quando a saída r for 0 o z_out = 1.

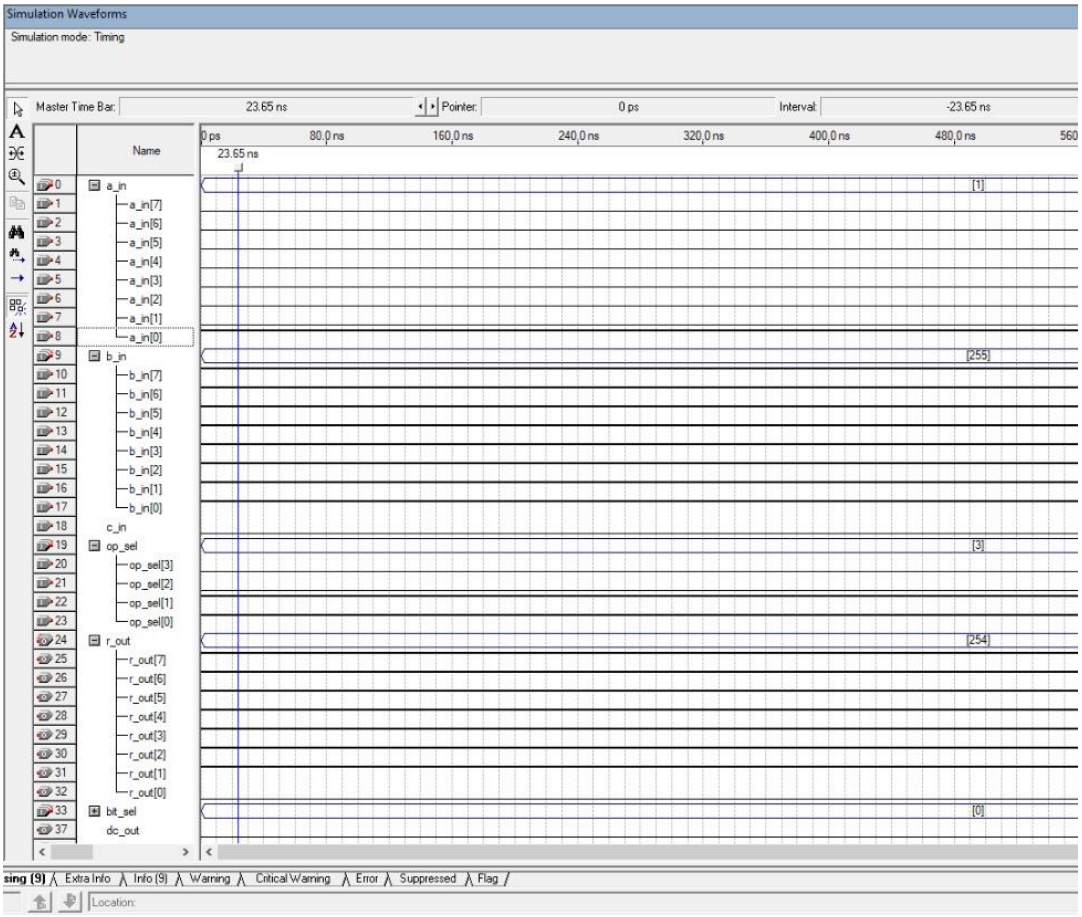

Figura 14: Simulação da operação COM.

A Figura 14 retrata a operação COM. Essa operação resulta na NOT A, invertendo os bits selecionados. Sendo A = 0000 0001, r_out = 1111 1110 e Z = 0. Quando a saída r for 0 o z_out = 1.

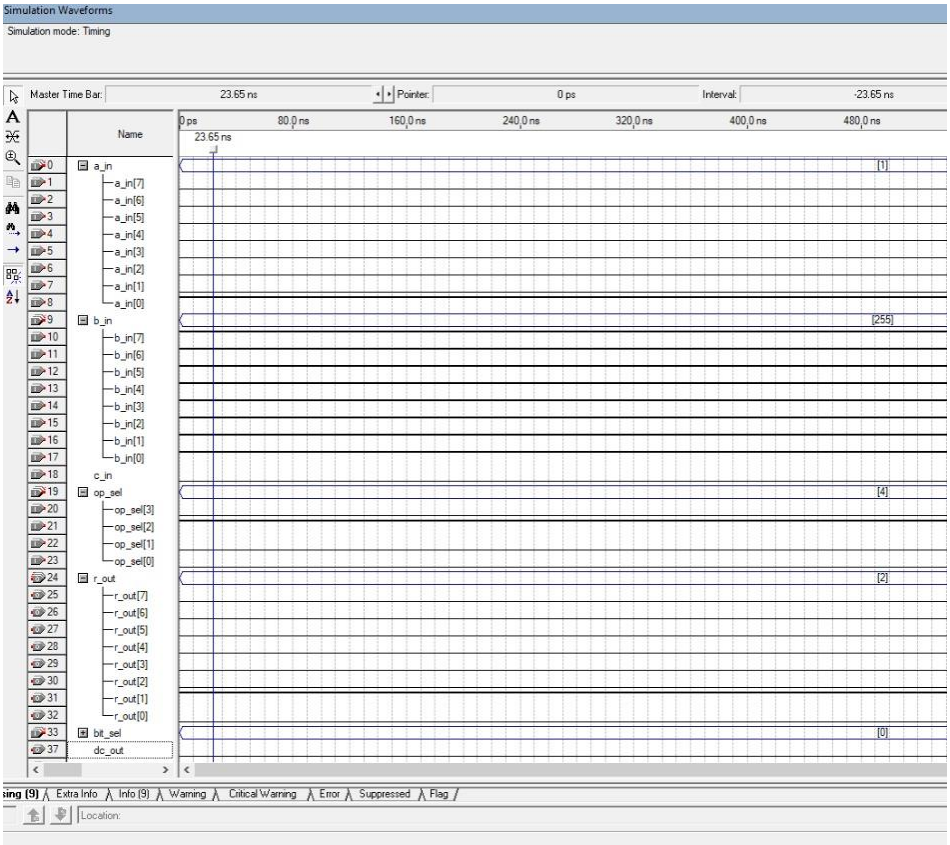

Figura 15: Simulação da operação INC.

A Figura 15 retrata a operação INC. Essa operação incrementa +1 no a_in. Sendo A = 0000 0001 + 1 o r_out = 0000 0010, pois houve a soma de mais 1 no número binário. Quando a saída r for 0 o z_out = 1.

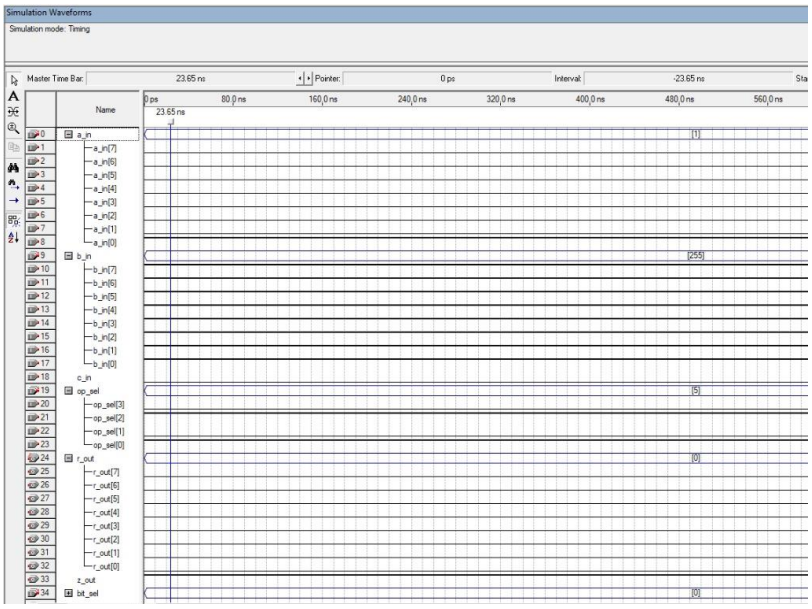

Figura 16: Simulação da operação DEC.

A Figura 16 retrata a operação DEC. Essa operação decrementa -1 no a_in. Sendo A = 0000 0001 - 1 o r_out = 0000 0000, pois houve a subtração de menos 1 no

número binário. Neste caso podemos ver que a saída r_out = 0 resultando no z_out = 1.

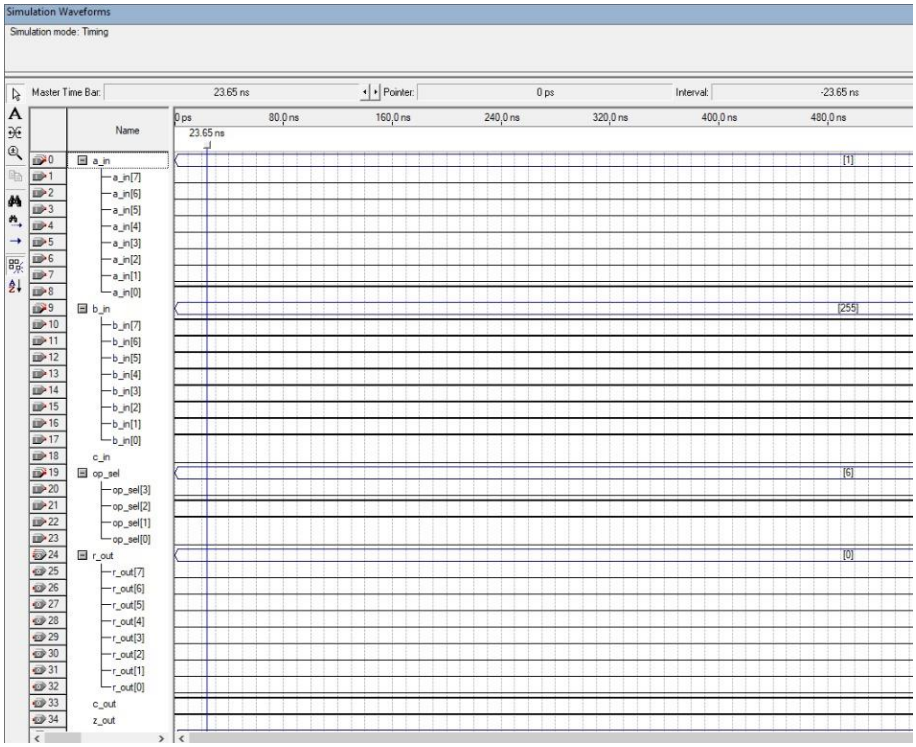

Figura 17: Simulação da operação ADD.

A Figura 17 retrata a operação ADD. Em ADD usaremos o operador + para realização de a_in + b_in, como houve carry c_out == 1, como r_out == 0 resultou em z_out ==1, respectivamente.

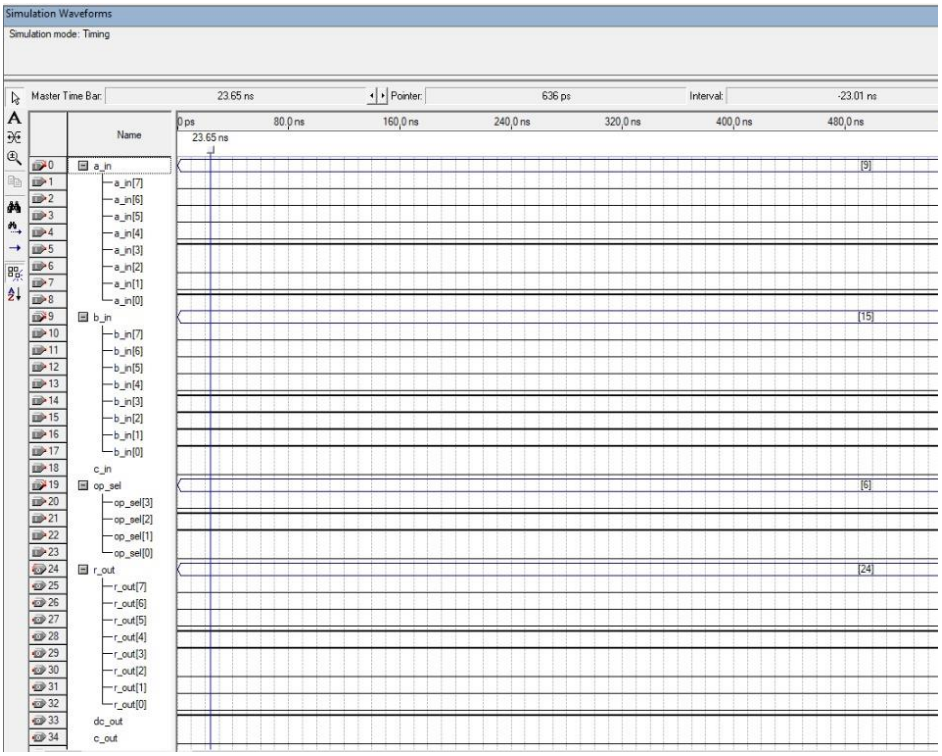

Figura 18: Simulação da operação add com carry no primeiro nibble.

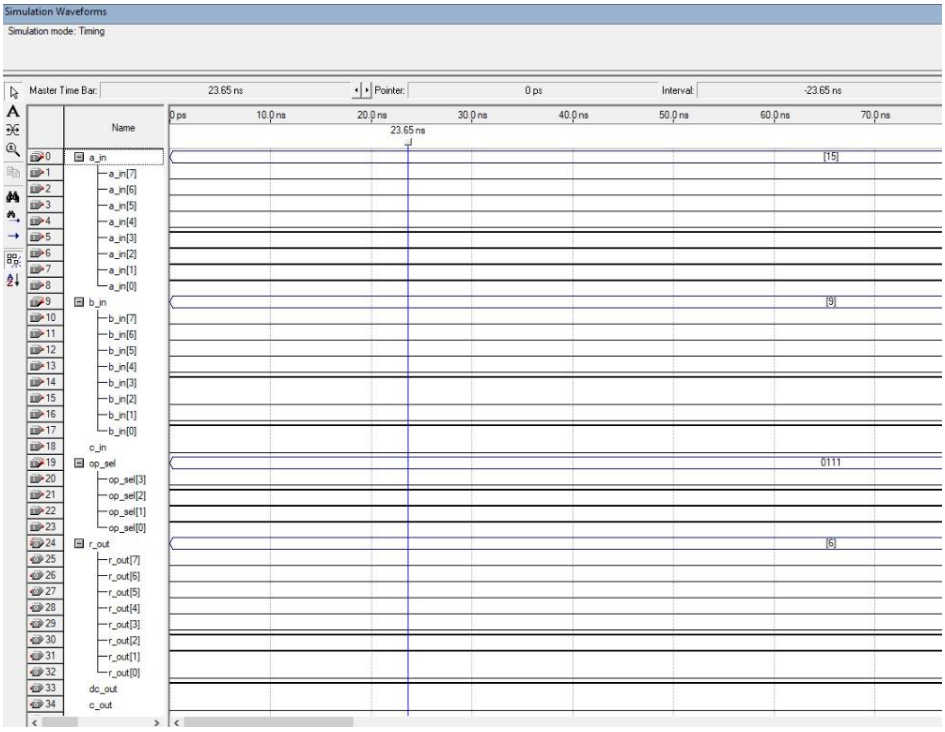

Figura 19: Simulação da operação SUB.

No SUB usaremos o operador - para realização de a_in - b_in, sendo a_in = 0000 1111 e b_in = 0000 1001. O resultado será r_out = 0000 0110 e z_out = 0. Quando a saída r for 0 o z_out = 1.

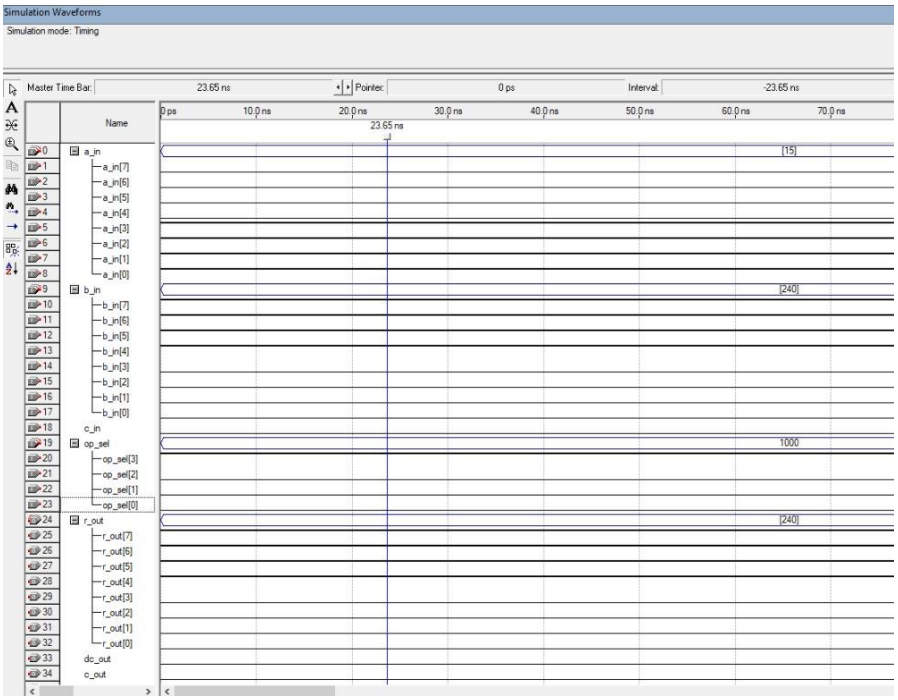

Figura 20: Simulação da operação SWAP.

No SWAP permuta os nibbles, o que estava no vetor de 0 - 3 passa a estar na saida r_out de 7 – 4.

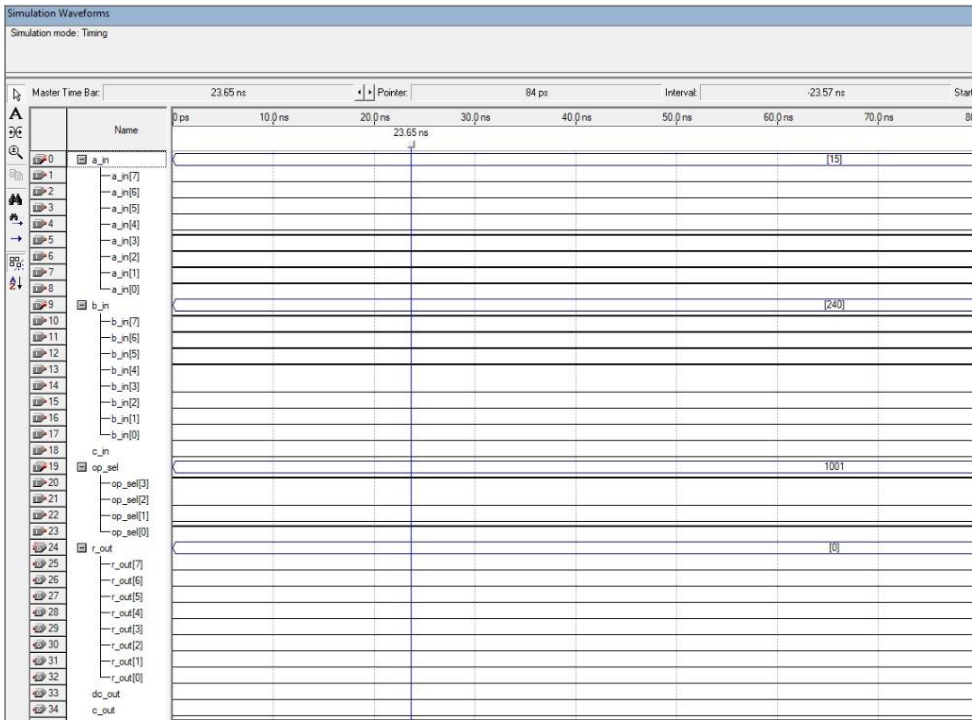

Figura 21: Simulação da operação CLR.

No CLR toda a saída r_out será limpada independente da entrada a_in e b_in e z_out = 1.

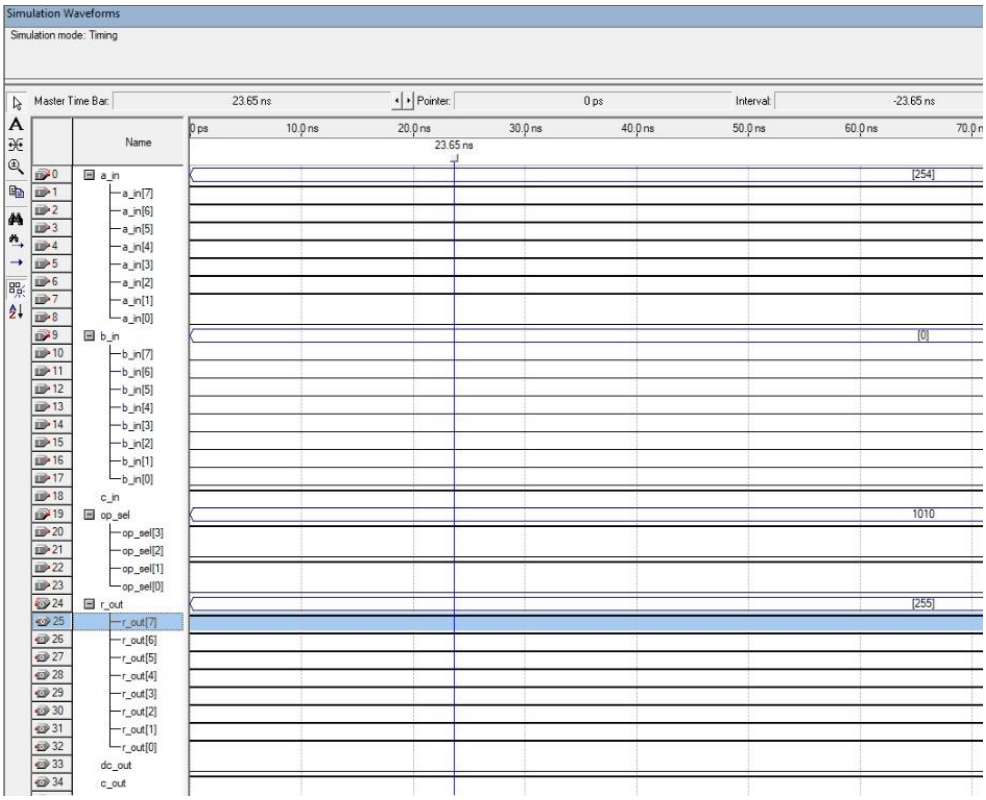

Figura 22: Simulação da operação RR.

Na rotação para direita, c_in [7], a_in [6-0], formando assim a saída de 8 bits, no a_in.

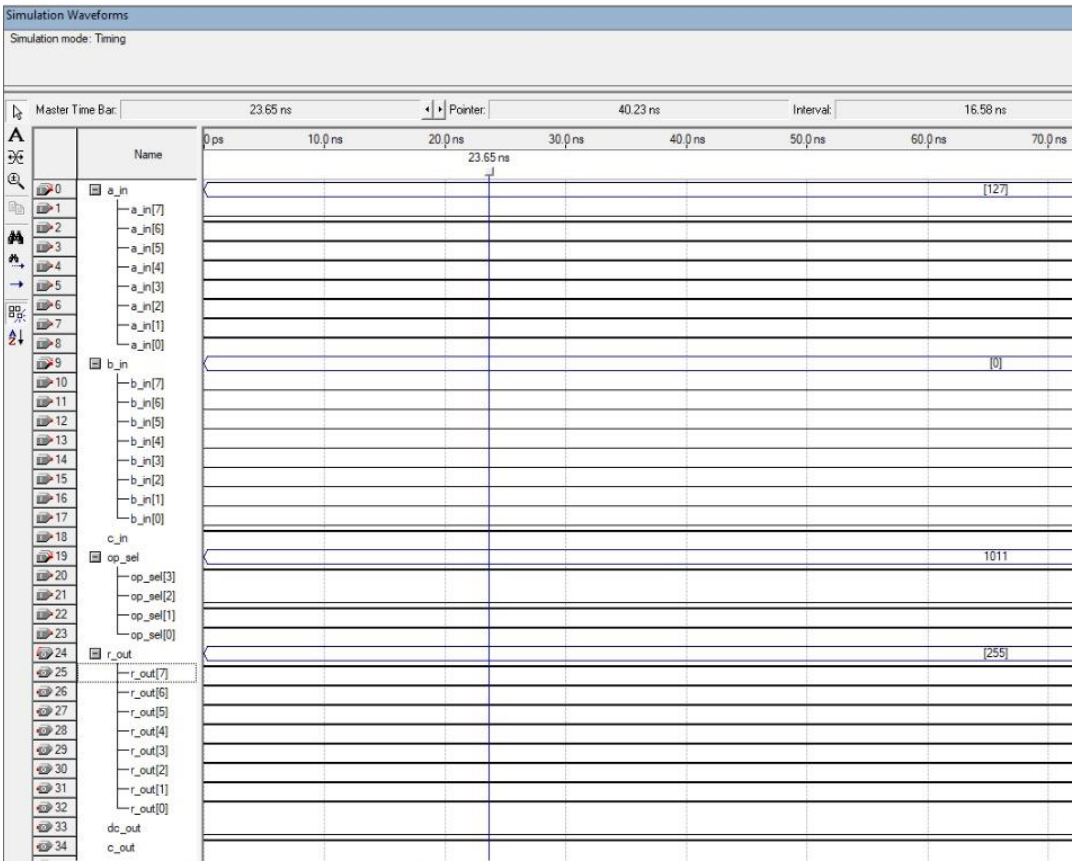

Figura 23: Simulação da operação RL.

Na rotação para esquerda, c_out = a_in[7].

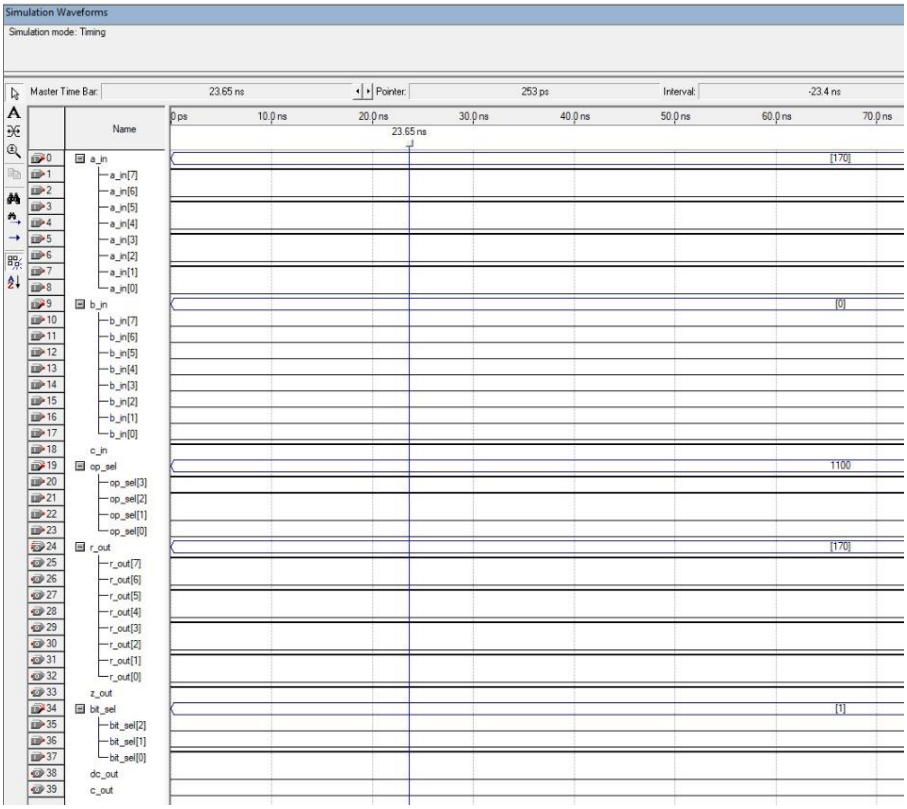

Figura 24: Simulação da operação BS.

Como o bit apontado pelo bit_sel = 1, a posição 1 do r_out permaneceu como high, semelhante há como estava em a_in.

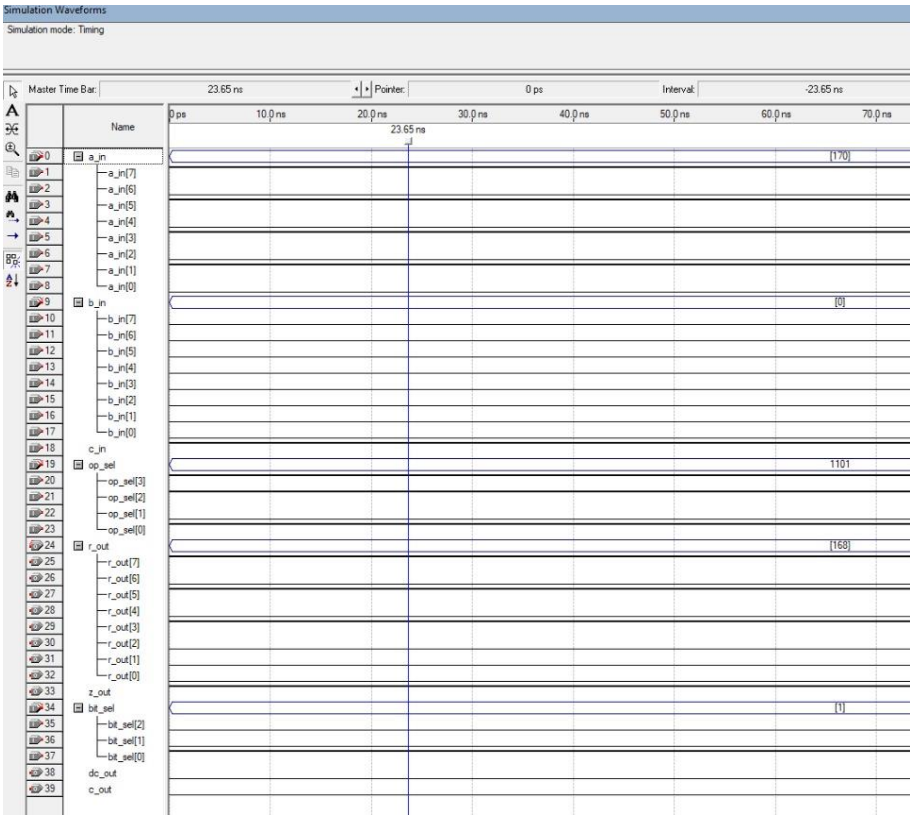

Figura 25: Simulação da operação BC.

Permanecendo as mesmas entradas do exemplo de bit_set (1100), quando trocar para a operação de bit_clear(1101), o elemento da posição setado pelo bit_set, que nesse caso era igual a 1, teria que se tornar 0 em r_out. a simulação comprova isso

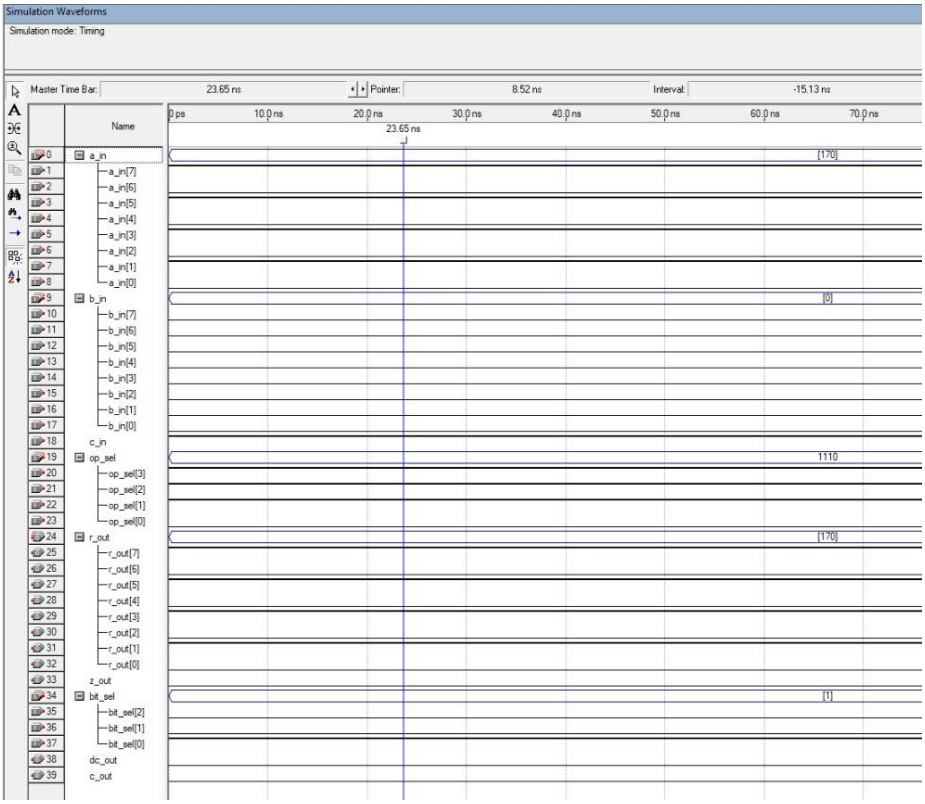

Figura 26: Simulação da operação PASS A.

Na operação bypass A, r_out recebe os valores de A. Quando a saída r for 0 o z_out = 1.

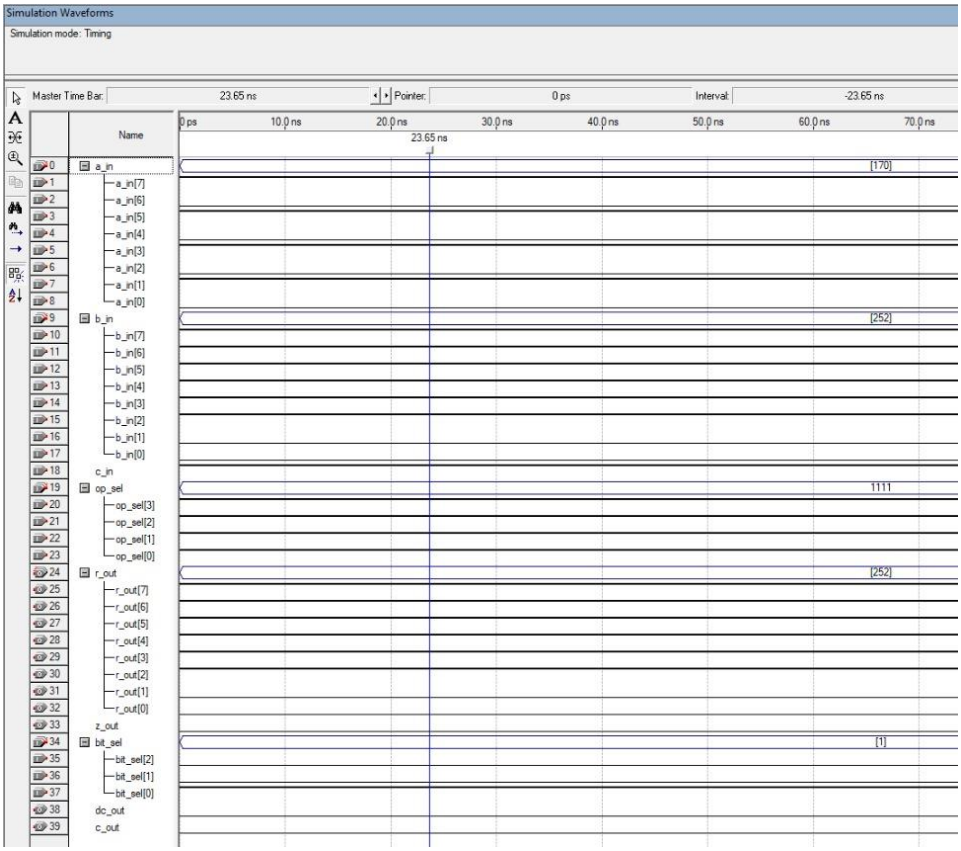

Figura 27: Simulação da operação PASS B.

Na operação by-pass B, r_out recebe os valores de B. Quando a saída r for 0 o z_out = 1.

III - CONCLUSÃO:

Ao desenvolver uma unidade lógica e aritmética (ALU) por meio da descrição em VHDL, esta experiência demonstrou claramente como o uso dessas ferramentas oferece suporte fundamental para a criação de protótipos e testes. Além disso, destacou a versatilidade da descrição de hardware, que não se limita apenas ao projeto de uma ULA, mas revela seu potencial em outras áreas, como a concepção de circuitos específicos com desempenho superior em comparação com a execução em microprocessadores genéricos.

Assim, este trabalho validou a eficácia do VHDL na descrição de uma ULA e na simulação por meio do ambiente de desenvolvimento Quartus II.

IV - REFERÊNCIAS:

Foram adotadas como fontes de referência o material didático fornecido pelo professor da disciplina, bem como o documento de especificação do projeto da ULA.